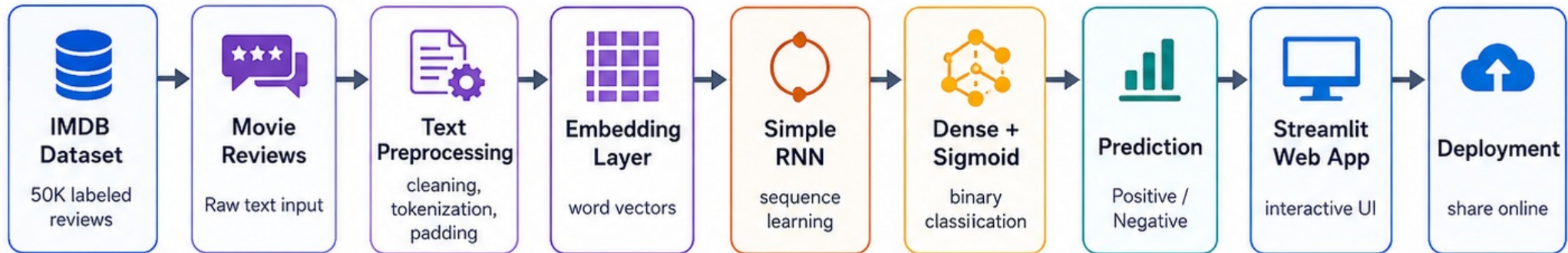


IMDB Sentiment Analysis with Simple RNN

End-to-End Deep Learning Project Workflow

Movie Reviews • Text Processing • Simple RNN • Sentiment Prediction • Web App Deployment



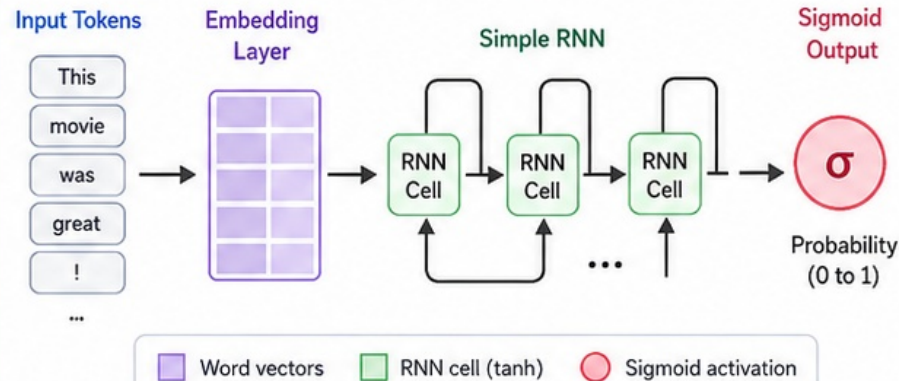
Sample Review

“ This movie was emotional, engaging, and beautifully acted. ”

✓ Predicted Sentiment: **Positive**

Confidence: 93%

Model Architecture (Overview)



Web App Preview

Enter a movie review

Type your review here...

Predict Sentiment

😊 **Positive**

📘 Built for deployment



Model Core

$$e_t = \text{Embedding}(x_t)$$

$$h_t = \tanh(W_x e_t + W_h h_{t-1} + b)$$

$$\hat{y} = \sigma(W_o h_t + c)$$

x_t : input token at time t h_t : hidden state $\hat{y}$: predicted output
 σ : sigmoid activation



Tools Used

- Python
- TensorFlow / Keras
- Google Colab
- Streamlit
- GitHub



Project Outcome

- ✓ Sentiment classification from text reviews
- ✓ End-to-end training workflow
- ✓ Deployable web application

Inside Simple RNN: From Words to Sentiment

Word Representation, Sequence Learning, and Sentiment Prediction

One-Hot Encoding • Word Embedding • Simple RNN • Sigmoid Output

1 One-Hot Encoding Problem



Words are represented as sparse vectors in a large vocabulary.

Vocabulary size

$$|V| = 10,000$$



Problems: high dimensional, sparse matrix, weak semantic meaning, can lead to overfitting.

man (ID = 5800)

boy (ID = 2000)

index
5800

0
0
0
⋮
1
0
0
⋮
0

index
2000

0
0
0
⋮
1
0
0
⋮
0

2 Word Embedding Solution

The embedding layer converts discrete words into dense feature vectors.

Embedding Layer / Word2Vec-style dense vectors

Feature dimension = 300

Vocabulary size $|V| = 10,000$

Feature Axes	Boy (ID=2000)	Girl (ID=5000)	King (ID=6000)	Queen (ID=7000)	Apple (ID=1000)	Mango (ID=8000)
♂ Gender	-1.00	1.00	-0.92	0.92	0.00	0.10
👑 Royal	0.01	0.03	0.95	0.96	-0.02	0.01
👤 Age	0.25	0.20	-0.10	-0.12	-0.30	-0.25
🍏 Food	-0.05	-0.04	-0.06	-0.05	0.98	0.97
⋮	⋮	⋮	⋮	⋮	⋮	⋮



Dense vectors capture semantic relationships better than one-hot encoding.

3 Sequence Learning with Simple RNN



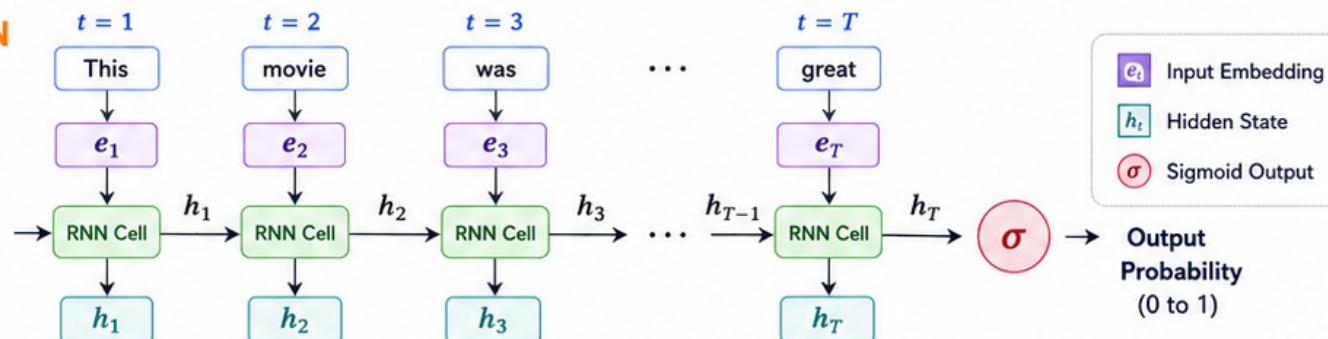
The RNN reads the review word by word and updates its hidden state to capture sequence information.

Input Tokens

Input Embeddings e_t

RNN Cells

Hidden States h_t



Σ Core Equations

$$e_t = \text{Embedding}(x_t)$$

$$h_t = \tanh(W_x e_t + W_h h_{t-1} + b)$$

$$\hat{y} = \sigma(W_o h_T + c)$$

x_t = input token at time t e_t = embedding vector

h_t = hidden state $\hat{y}$ = predicted output

σ = sigmoid activation



Example Prediction

“

This movie was excellent and engaging.

”

Positive
= 0.93

Negative
= 0.07



Prediction: Positive



Key Takeaways

- ✓ One-hot vectors are sparse and inefficient
- ✓ Embeddings provide dense feature representations
- ✓ Simple RNN processes text sequentially
- ✓ Final sigmoid layer outputs sentiment probability